UAC
Universidad Autónoma de Campeche
"Del enigma sin albas a triángulos de luz"

Facultad de Ingeniería

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

FAULTAD DE INGENIERÍA

REDES DE COMPUTADORAS

ITS

REPORTE

PRÁCTICA: MEDICIÓN DE TIEMPO DE EJECUCIÓN CON FLUJOS CON Y SIN BUFFER

- ALUMNO: DANIEL FRANCISCO BEYTIA CHI

- GRADO: 5 SEMESTRE

- GRUPO: A -MATRICULA: 75469

-

Objetivo

Generar y leer archivos de texto y binarios en Java con las clases FileWriter, BufferedWriter, FileReader, BufferedReader, FileOutputStream, BufferedOutputStream, FileInputStream y BufferedInputStream, y medir el tiempo de cada operación con System.nanoTime() para comparar los flujos con y sin buffer.

Desarrollo

Se creó la clase EscrituraBuffered (paquete pract01) con ocho métodos, uno por cada flujo. En los flujos de texto se escribe y se lee un carácter por llamada; en los binarios, un byte por llamada. Así la única diferencia entre cada par de métodos es el uso del buffer. Los buffers se vacían con flush() antes de detener el cronómetro.

El programa se ejecutó desde la terminal con java pract01.EscrituraBuffered n, con n = 100,000, 1,000,000 y 10,000,000, tres veces por cada valor. Las pruebas se hicieron en un equipo con Windows, y los tiempos medidos no incluyen la apertura ni el cierre del archivo.

Resultados

Tabla con n = 100,000

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	100,000	71.227	56.843	55.955	**61.342**
BufferedWriter	100,000	21.540	14.534	15.353	**17.142**
FileReader	100,000	44.106	53.134	47.263	**48.168**
BufferedReader	100,000	15.093	17.680	12.264	**15.012**
FileOutputStream	100,000	1,044.657	839.428	1,034.659	**972.915**
BufferedOutputStream	100,000	11.831	11.607	9.757	**11.065**
FileInputStream	100,000	421.254	384.993	425.197	**410.481**
BufferedInputStream	**100,000**	**12.705**	**12.185**	**9.649**	**11.513**

Tabla con n = 1,000,000

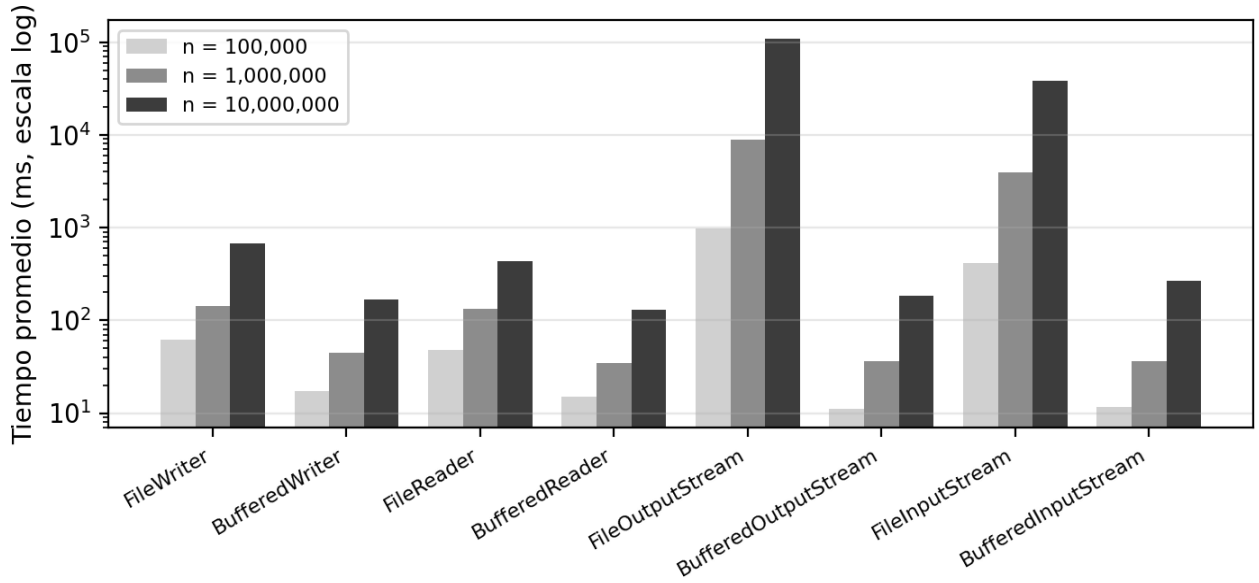
Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	1,000,000	156.794	131.483	135.473	**141.250**
BufferedWriter	1,000,000	49.044	41.922	44.367	**45.111**
FileReader	1,000,000	137.345	134.515	129.034	**133.631**
BufferedReader	1,000,000	28.710	43.822	30.845	**34.459**
FileOutputStream	1,000,000	9,714.694	7,400.256	9,734.465	**8,949.805**
BufferedOutputStream	1,000,000	37.518	40.327	30.189	**36.011**
FileInputStream	1,000,000	3,931.016	3,870.148	3,925.163	**3,908.776**
BufferedInputStream	**1,000,000**	**36.989**	**39.884**	**32.590**	**36.488**

Tabla con n = 10,000,000

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	10,000,000	666.238	676.603	679.166	**674.002**
BufferedWriter	10,000,000	183.272	161.366	157.378	**167.339**
FileReader	10,000,000	456.575	429.267	428.816	**438.219**
BufferedReader	10,000,000	134.546	129.954	129.606	**131.369**
FileOutputStream	10,000,000	116,032.836	108,349.284	103,379.563	**109,253.894**
BufferedOutputStream	10,000,000	229.593	138.146	186.139	**184.626**
FileInputStream	10,000,000	40,376.978	36,890.787	37,223.978	**38,163.914**
BufferedInputStream	**10,000,000**	**503.214**	**143.803**	**146.474**	**264.497**

Nota: En la ejecución 1 de BufferedInputStream con n = 10,000,000 se midieron 503.214 ms, mucho más que las otras dos (cerca de 145 ms). Es probable que se deba al calentamiento de la JVM (compilación JIT) o a actividad del sistema operativo. Se dejó en el promedio, por eso éste sale más alto de lo habitual.

Promedios de los tres valores de n (escala logarítmica)



Análisis y respuestas

¿Qué diferencia observaste entre usar flujos con buffer y sus equivalentes directos? ¿Por qué ocurre?

Los flujos con buffer fueron más rápidos en todos los casos. Con n = 10,000,000, BufferedWriter fue unas 4 veces más rápido que FileWriter y BufferedReader unas 3 veces más rápido que FileReader. En binario la diferencia fue mucho mayor: BufferedOutputStream escribió en 184.626 ms contra 109,253.894 ms de FileOutputStream (unas 592 veces), y BufferedInputStream leyó en 264.497 ms contra 38,163.914 ms de FileInputStream (unas 144 veces). Esto ocurre porque cada write() o read() sin buffer provoca una llamada al sistema operativo, que es costosa. Con buffer, los datos se acumulan en memoria y se leen o escriben en bloques, así que el número de accesos al disco baja de millones a unos cuantos miles.

¿Cómo varía el tiempo al aumentar n? ¿Es lineal? ¿Se comporta igual en lectura que en escritura?

En los flujos sin buffer de bytes el crecimiento es cercano a lineal (entre 10 y 12 veces): al pasar de 1,000,000 a 10,000,000 (10 veces más datos), FileOutputStream pasó de 8,949.805 a 109,253.894 ms (unas 12 veces) y FileInputStream de 3,908.776 a 38,163.914 ms (unas 10 veces). En los flujos con buffer el crecimiento es menor a 10 veces entre n = 100,000 y 1,000,000, porque con pocos datos pesan más los costos fijos de la JVM, como el calentamiento y la apertura de archivos. La escritura fue más lenta que la lectura en binario sin buffer: escribir 10,000,000 de bytes tardó cerca de 109 s y leerlos cerca de 38 s, es decir, unas 2.9 veces más. En los flujos con buffer, lectura y escritura tardaron tiempos parecidos.

¿Hay diferencia entre trabajar con caracteres y con bytes? ¿A qué se debe?
Sí. Sin buffer, los flujos de texto resultaron mucho más rápidos que los de bytes: con n = 10,000,000, FileWriter tardó 674.002 ms y FileOutputStream 109,253.894 ms. La razón es que FileWriter y FileReader no son realmente directos: internamente usan un codificador y decodificador que mantienen un buffer propio de 8 KB, así que no hacen una llamada al sistema por carácter. FileOutputStream y FileInputStream sí la hacen por cada byte. Con buffer, texto y binario quedan en el mismo orden de magnitud (de unos cientos de milisegundos con 10,000,000 de elementos). El texto tiene el costo adicional de convertir caracteres según el charset por defecto, mientras que los bytes se escriben tal cual.

¿Qué tamaño tiene el buffer por defecto? ¿Puede cambiarse?
BufferedOutputStream y BufferedInputStream usan un buffer de 8,192 bytes (8 KB) por defecto. BufferedWriter y BufferedReader usan uno de 8,192 caracteres. En los cuatro casos se puede cambiar usando el constructor que recibe el tamaño, por ejemplo new BufferedReader(fr, 65536).

¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no? ¿Qué implicación tiene al leer texto?
BufferedReader trabaja con caracteres, así que puede reconocer los saltos de línea y devolver una línea completa como String. BufferedInputStream solo maneja bytes sin interpretación, y no sabe qué es una línea ni qué codificación tienen. Al leer texto conviene usar BufferedReader, porque lee por líneas y además decodifica correctamente los caracteres; con BufferedInputStream habría que reconstruir las líneas y la codificación a mano.

Conclusión

Esta práctica me mostró con datos propios que el buffer es una de las optimizaciones más sencillas y con más efecto al trabajar con archivos en Java. Con 10,000,000 de bytes, escribir sin buffer tomó casi dos minutos y con buffer menos de un cuarto de segundo, usando el mismo código y cambiando solo una clase. En texto la mejora fue menor, de unas 4 veces, porque FileWriter y FileReader ya usan un buffer interno. Concluyo que en cualquier programa que lea o escriba muchos datos conviene envolver los flujos en sus versiones con buffer, ya sea de texto o binario, y cerrarlos con try-with-resources para que se vacíen correctamente. También aprendí que las mediciones simples tienen variaciones, como la ejecución atípica de BufferedInputStream, y que por eso hay que repetirlas y promediarlas.